Laporan UTS Graf Sains Data

Nama: Jihan Amanda Siagian

NIM: 23031554184

Mata Kuliah: Graf Sains Data

Pendahuluan

Tugas ini menggunakan dataset RetailRocket, yaitu data interaksi pengunjung pada sebuah toko online. File yang dipakai adalah events.csv yang berisi 2.756.101 baris dengan tiga jenis event, yaitu view, addtocart, dan transaction. Data ini dimuat ke dalam database graf Neo4j dengan dua jenis simpul, yaitu Visitor dan Item, serta tiga jenis relasi dari Visitor ke Item, yaitu VIEWED, ADDED_TO_CART, dan BOUGHT. Setiap relasi menyimpan timestamp, dan khusus relasi BOUGHT juga menyimpan transactionId.

Proses impor dilakukan per 1.000 baris. Sebelumnya dibuat constraint unik untuk visitorId dan itemId agar tidak ada simpul ganda. Setelah impor, database berisi 1.407.580 simpul Visitor.

A. Wrangling dan EDA

Soal 1. Penentuan Periode Waktu

Rentang data RetailRocket adalah 3 Mei 2015 hingga 18 September 2015. Dari rentang tersebut saya memilih dua periode berikut.

Fold	Tanggal	Lama	Fungsi
P1 (training)	25 Mei 2015 s.d. 13 Agustus 2015	81 hari	membangun VSM, similarity, dan KNN
P2 (testing)	14 Agustus 2015 s.d. 3 September 2015	21 hari	data uji untuk evaluasi rekomendasi

P2 dimulai tepat sehari setelah P1 berakhir sehingga tidak ada data yang tumpang tindih. Karena kolom timestamp pada data berbentuk epoch milidetik, batas periode juga diubah ke epoch milidetik. P1 berada di antara 1432512000000 dan 1439510399999, sedangkan P2

berada di antara 1439510400000 dan 1441324799999.

Untuk membedakan kekuatan setiap interaksi, saya memberi bobot 1 untuk view, 3 untuk addtocart, dan 5 untuk transaction.

Soal 2. Filter Timestamp dan Penyimpanan ke CSV

Data dibaca dengan pandas, kemudian dipilih baris yang timestamp-nya masuk P1 atau P2. Setiap baris diberi kolom fold dan weight. Baris yang sama persis (visitor, item, event, dan timestamp sama) dihapus agar jumlahnya sesuai dengan relasi di Neo4j yang dibuat dengan MERGE.

```
in_p1 = events["timestamp"].between(*P1_MS)
in_p2 = events["timestamp"].between(*P2_MS)

filtered = events[in_p1 | in_p2].copy()
filtered["fold"] = np.where(in_p1[in_p1 | in_p2], "P1", "P2")
filtered["weight"] = filtered["event"].map(EVENT_WEIGHT)
filtered = filtered.drop_duplicates(["visitorid", "itemid", "event",
"timestamp"])

filtered.to_csv("events_filtered_20150525_20150903.csv", index=False)
```

Hasil filter disimpan ke file events_filtered_20150525_20150903.csv dengan jumlah 2.056.946 baris. Ringkasan isinya dapat dilihat pada tabel berikut.

Keterangan	P1	P2
Jumlah interaksi	1.685.016	371.930
Visitor unik	871.840	210.644
Item unik	190.368	92.238
view	1.629.117	358.951
addtocart	42.021	9.797
transaction	13.878	3.182
Interaksi pertama	25-05-2015 00:00:00	14-08-2015 00:00:00
Interaksi terakhir	13-08-2015 23:59:56	03-09-2015 23:59:57

Soal 3. Komposisi Subgraph

Subgraph di Neo4j dibentuk dengan menandai relasi yang masuk ke dalam periode. Relasi pada P1 atau P2 diberi properti fold dan weight, sedangkan relasi di luar periode tidak memiliki fold. Query yang digunakan:

```
MATCH ()-[r:VIEWED|ADDED_TO_CART|BOUGHT]->()
CALL (r) {
  SET r.fold = CASE
        WHEN r.timestamp >= $p1_start AND r.timestamp <= $p1_end THEN 'P1'
        WHEN r.timestamp >= $p2_start AND r.timestamp <= $p2_end THEN 'P2'
        ELSE null END,
      r.weight = CASE type(r) WHEN 'VIEWED' THEN 1 WHEN 'ADDED_TO_CART' THEN
3 ELSE 5 END
} IN TRANSACTIONS OF 50000 ROWS
```

Query ini membaca semua relasi interaksi lalu mengisi fold sesuai timestamp. Jika timestamp di luar P1 dan P2, fold diisi null sehingga propertinya hilang. Bobot diisi berdasarkan jenis relasi. Karena jumlah relasinya jutaan, update dipecah per 50.000 baris dengan IN TRANSACTIONS supaya tidak kehabisan memori. Proses ini selesai dalam 33,2 detik.

Komposisi subgraph kemudian dihitung dari Python menggunakan driver Neo4j dengan tiga query berikut.

```
// jenis dan jumlah simpul
MATCH (n)-[r:VIEWED|ADDED_TO_CART|BOUGHT]-()
WHERE r.fold IS NOT NULL
WITH DISTINCT n
RETURN labels(n)[0] AS jenis_simpul, count(*) AS jumlah

// simpul dan relasi per fold
MATCH (v:Visitor)-[r:VIEWED|ADDED_TO_CART|BOUGHT]->(i:Item)
WHERE r.fold IS NOT NULL
RETURN r.fold AS fold, count(DISTINCT v) AS visitor, count(DISTINCT i) AS
item, count(r) AS relasi

// relasi per jenis per fold
MATCH ()-[r:VIEWED|ADDED_TO_CART|BOUGHT]->()
WHERE r.fold IS NOT NULL
RETURN r.fold AS fold, type(r) AS jenis_relasi, count(*) AS jumlah
```

Pada query pertama, setiap simpul yang terhubung dengan relasi bertanda fold diambil tanpa melihat arah relasi. WITH DISTINCT dipakai agar satu simpul tidak terhitung berkali-kali, lalu jumlahnya dikelompokkan menurut label. Query kedua menghitung visitor dan item yang berbeda di masing-masing fold. Query ketiga menghitung banyaknya relasi untuk setiap jenis interaksi.

Hasil query pertama menunjukkan subgraph terdiri dari 1.065.669 simpul Visitor dan 208.339 simpul Item, sehingga totalnya 1.274.008 simpul. Jumlah ini sama dengan jumlah visitor dan item unik pada file CSV hasil filter. Hasil query kedua dan ketiga juga sama dengan tabel pada Soal 2, yaitu 1.685.016 relasi pada P1 dan 371.930 relasi pada P2. Sebagian besar relasi adalah VIEWED, sedangkan BOUGHT hanya kurang dari 1%.

Soal 4. Vector Space Model untuk User

Dalam VSM User, satu visitor dianggap sebagai satu vektor. Panjang vektornya sama dengan banyaknya item pada P1, dan isi setiap elemennya adalah jumlah bobot interaksi visitor itu dengan item tersebut. Jika visitor tidak pernah menyentuh suatu item, nilainya 0.

Data P1 dikelompokkan per pasangan visitor dan item, lalu bobotnya dijumlahkan dan disusun menjadi matriks sparse CSR. Matriks sparse dipilih karena hampir semua elemennya bernilai 0. Untuk proyeksi, dibuat juga matriks biner B (semua nilai bukan nol diganti 1). Proyeksi User didapat dari perkalian B dengan transposnya, sehingga dua visitor terhubung jika pernah menyentuh item yang sama dan bobot sisinya adalah jumlah item yang sama.

```
ui = train.groupby(["visitorid", "itemid"], as_index=False)["weight"].sum()
X = sp.csr_matrix(
    (ui["weight"], (user_index.get_indexer(ui["visitorid"]),
item_index.get_indexer(ui["itemid"]))),
    shape=(len(user_index), len(item_index)),
)
B = X.copy()
B.data[:] = 1

P_user = (B @ B.T).tocsr()
P_user.setdiag(0)
P_user.eliminate_zeros()
```

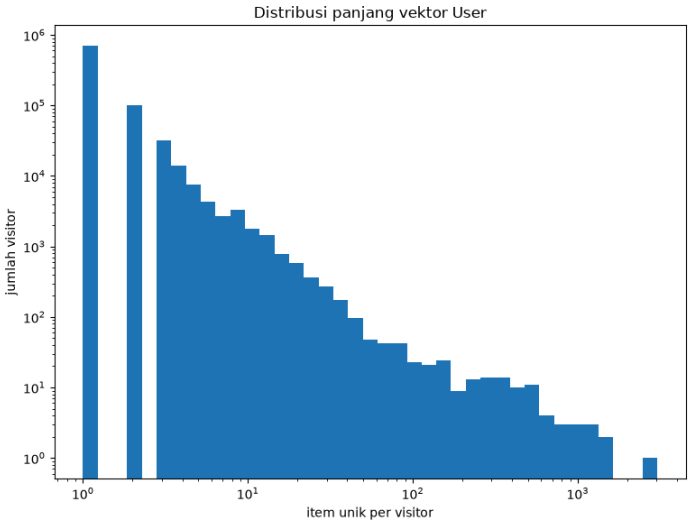
Karakteristik VSM User yang diperoleh:

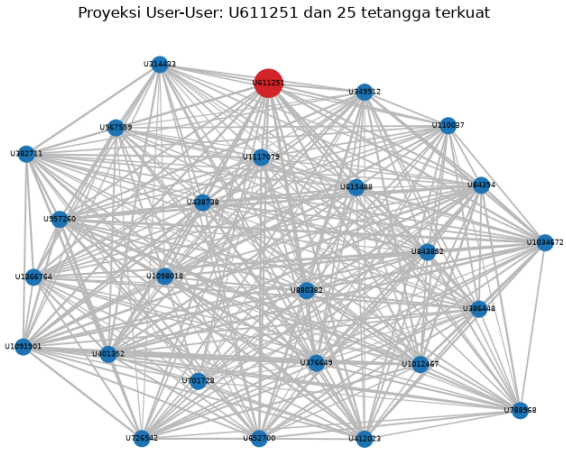
- Shape matriks 871.840 x 190.368, artinya ada 871.840 vektor dengan dimensi 190.368.

- Jumlah elemen bukan nol 1.313.544, dengan density sekitar 0,000008.
- Rata-rata satu visitor hanya menyentuh 1,51 item, median 1 item, dan nilai terbesar 3.028 item.
- Sebanyak 80,3% visitor hanya memiliki satu elemen bukan nol.
- Dalam bentuk sparse matriks ini hanya memakai 14,0 MB, sedangkan jika dense akan memakai sekitar 663,9 GB.

Contoh isi vektor: visitor 1 hanya berisi item 72028 dengan bobot 1, sedangkan visitor 2 berisi item 216305 (2), 259884 (1), 325215 (3), dan 342816 (2).

Proyeksi User menghasilkan graf dengan 871.840 simpul dan 36.093.009 sisi, rata-rata degree 82,8, dan 5,3% simpul tidak memiliki tetangga. Proyeksi ini memakai memori 581,0 MB dan dihitung dalam 2,9 detik.





Histogram di sebelah kiri memperlihatkan bahwa jumlah visitor turun tajam ketika jumlah item bertambah. Ada sebagian kecil visitor yang menyentuh ribuan item dan kemungkinan merupakan bot. Plot di sebelah kanan dibuat dengan networkx dan menampilkan satu visitor beserta 25 tetangga dengan bobot terbesar. Visitor pusat diambil dari visitor yang riwayatnya paling banyak 20 item agar yang terpilih bukan bot. Plot tersebut terlihat sangat rapat karena banyak visitor yang terhubung hanya dari satu item populer yang sama.

Soal 5. Vector Space Model untuk Item

VSM Item adalah kebalikan dari VSM User. Satu item dianggap sebagai satu vektor yang panjangnya sama dengan banyaknya visitor pada P1. Matriksnya cukup didapat dengan mentranspos matriks User. Proyeksi Item didapat dari perkalian transpos B dengan B, sehingga dua item terhubung jika pernah disentuh oleh visitor yang sama.

```
X_item = X.T.tocsr()
B_item = B.T.tocsr()

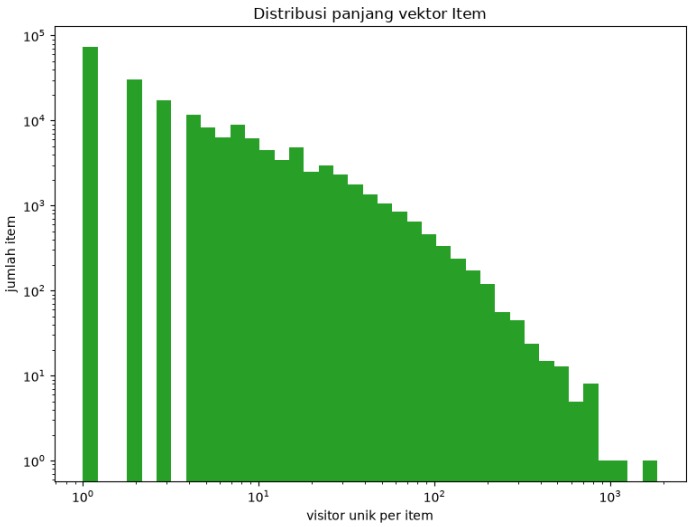
P_item = (B_item @ B_item.T).tocsr()
P_item.setdiag(0)
P_item.eliminate_zeros()
```

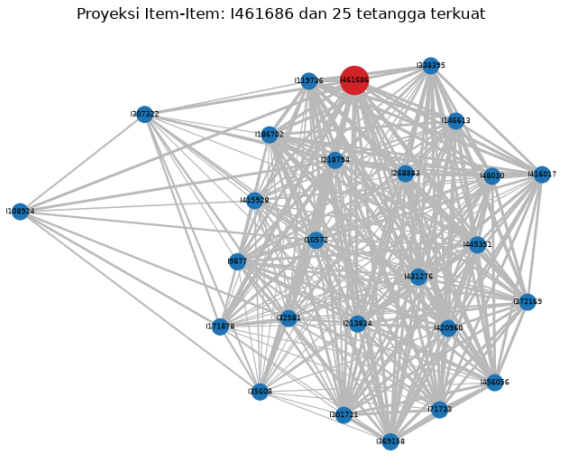
Karakteristik VSM Item yang diperoleh:

- Shape matriks 190.368 x 871.840, artinya ada 190.368 vektor dengan dimensi 871.840.
- Jumlah elemen bukan nol sama dengan VSM User, yaitu 1.313.544, dan density juga sekitar 0,000008.
- Rata-rata satu item disentuh oleh 6,90 visitor, median 2 visitor, dan nilai terbesar 1.846 visitor.
- Item yang hanya disentuh satu visitor sebanyak 38,5%.
- Memori sparse yang dipakai 11,3 MB.

Contoh isi vektor: item 4 hanya disentuh visitor 1042455, sedangkan item 15 disentuh oleh banyak visitor, termasuk visitor 724401 dan 989647 yang masing-masing berbobot 2.

Proyeksi Item menghasilkan graf dengan 190.368 simpul dan 17.038.027 sisi, rata-rata degree 179,0, dan 37,6% simpul tidak memiliki tetangga. Proyeksi ini memakai memori 273,4 MB dan dihitung dalam 1,0 detik.





Plot di sebelah kanan berpusat pada item 461686, yaitu item dengan total bobot interaksi paling besar di P1. Tetangga-tetangga item ini juga saling terhubung satu sama lain, sehingga terlihat seperti satu kelompok produk yang sering dilihat bersamaan.

Soal 6. Perbandingan Proyeksi User dan Item

Aspek	Proyeksi User	Proyeksi Item
Jumlah simpul	871.840	190.368
Jumlah sisi	36.093.009	17.038.027
Rata-rata elemen bukan nol per vektor	1,51	6,90
Vektor dengan satu elemen bukan nol	80,3%	38,5%
Simpul tanpa tetangga	5,3%	37,6%
Rata-rata degree	82,8	179,0
Memori proyeksi	581,0 MB	273,4 MB
Waktu hitung	2,9 detik	1,0 detik

Menurut saya proyeksi Item lebih baik untuk data ini.

Jika dilihat dari efisiensi, proyeksi Item memiliki simpul sekitar 4,6 kali lebih sedikit dan sisi kurang dari setengah proyeksi User. Memori yang dibutuhkan juga lebih kecil (273 MB dibanding 581 MB) dan waktunya lebih cepat (1,0 detik dibanding 2,9 detik). Selain itu, similarity antar item cukup dihitung sekali dan bisa dipakai untuk semua user. Hal ini penting karena visitor terus berganti; dari 871 ribu visitor di P1 hanya sekitar 16,8 ribu yang muncul lagi di P2.

Jika dilihat dari efektivitas, vektor item jauh lebih berisi dibanding vektor user (6,9 dibanding 1,5 elemen bukan nol). Karena 80% vektor user hanya berisi satu item, nilai cosine similarity antar user hampir selalu 0 atau 1 dan tidak banyak membantu membedakan user. Proyeksi User memang lebih sedikit simpul yang terisolasi, tetapi hubungannya sebagian besar hanya karena satu item populer yang sama. Pada proyeksi Item, simpul yang terisolasi adalah item yang hanya dibuka satu visitor, sementara sekitar 119 ribu item lainnya memiliki tetangga yang lebih bermakna.

Berdasarkan alasan tersebut, perhitungan similarity dan KNN pada bagian selanjutnya menggunakan proyeksi Item.

Soal 7. Perhitungan Similarity

Ukuran kemiripan yang dipakai adalah cosine similarity. Nilai similarity antara item a dan item b adalah hasil kali titik kedua vektor dibagi dengan hasil kali panjang kedua vektor:

sim(a, b) = (jumlah dari w(u,a) x w(u,b)) / (akar jumlah w(u,a)² x akar jumlah w(u,b)²)

dengan w(u,a) adalah bobot interaksi visitor u pada item a. Nilainya berada di antara 0 dan 1, dan semakin mendekati 1 berarti kedua item semakin sering disentuh oleh visitor yang sama.

Di Python, setiap vektor item dinormalisasi terlebih dahulu sehingga panjangnya 1. Setelah itu matriks similarity untuk semua pasangan item bisa didapat hanya dengan mengalikan matriks item dengan transposnya. Hasilnya berukuran 190.368 x 190.368 dengan 17.038.027 pasangan yang nilainya bukan nol.

```
X_item_norm = normalize(X_item, norm="l2", axis=1)
S = (X_item_norm @ X_item_norm.T).tocsr()
S.setdiag(0)
S.eliminate_zeros()
```

Record acuan yang saya pilih adalah item 461686, karena item ini memiliki total bobot interaksi paling besar di P1 dan disentuh oleh 760 visitor.

a. Query Cypher untuk 5 record dengan similarity tertinggi

Perhitungan yang sama juga dilakukan langsung di Neo4j menggunakan relasi pada P1 saja.

```
MATCH (ref:Item {itemId: $ref})<-[r:VIEWED|ADDED_TO_CART|BOUGHT]-(v:Visitor)
WHERE r.fold = 'P1'
WITH ref, v, sum(r.weight) AS wa
WITH ref, collect({v: v, wa: wa}) AS vecA, sqrt(sum(wa * wa)) AS normA
UNWIND vecA AS a
WITH ref, normA, a.v AS v, a.wa AS wa
MATCH (v)-[r2:VIEWED|ADDED_TO_CART|BOUGHT]->(other:Item)
WHERE r2.fold = 'P1' AND other <> ref
WITH ref, normA, other, v, wa, sum(r2.weight) AS wb
WITH ref, normA, other, sum(wa * wb) AS dot
CALL (other) {
  MATCH (other)<-[r3:VIEWED|ADDED_TO_CART|BOUGHT]-(u:Visitor)
  WHERE r3.fold = 'P1'
  WITH u, sum(r3.weight) AS wb
  RETURN sqrt(sum(wb * wb)) AS normB
}
RETURN ref.itemId AS source, other.itemId AS target, dot / (normA * normB) AS
similarity
ORDER BY similarity DESC, target ASC
LIMIT 5
```

Cara kerja query di atas adalah sebagai berikut. Mula-mula dicari semua visitor yang berinteraksi dengan item acuan, kemudian bobotnya dijumlahkan untuk mendapatkan panjang vektor item acuan (normA). Dari visitor-visitor tersebut dicari item lain yang juga mereka sentuh, lalu hasil kali bobotnya dijumlahkan sebagai dot product. Bagian CALL menghitung panjang vektor setiap item lain (normB). Terakhir, dot product dibagi normA dan normB, diurutkan dari yang terbesar, dan diambil lima teratas.

No	Item acuan	Item mirip	Similarity
1	461686	171878	0,390812
2	461686	218794	0,389686
3	461686	32581	0,359416
4	461686	10935	0,260603
5	461686	14656	0,260603

Hasil query Cypher ini sama persis dengan hasil perhitungan di Python, baik urutan item maupun nilai similarity-nya.

b. Menuliskan similarity ke graf

Kelima nilai similarity tersebut disimpan ke graf sebagai relasi baru SIMILAR_TO dari item acuan ke item mirip. Relasi ini menyimpan nilai score, metode yang dipakai (cosine), dan fold asal data (P1). Sebelum menulis, relasi SIMILAR_TO yang lama dihapus terlebih dahulu.

```
UNWIND $rows AS row
MATCH (a:Item {itemId: $ref}), (b:Item {itemId: row.target})
MERGE (a)-[s:SIMILAR_TO]->(b)
SET s.score = row.similarity, s.method = 'cosine', s.fold = 'P1'
```

Untuk mengecek hasilnya digunakan query berikut.

```
MATCH (a:Item {itemId: $ref})-[s:SIMILAR_TO]->(b:Item)
RETURN a.itemId AS source, b.itemId AS target, s.score AS similarity,
s.method AS method, s.fold AS fold
ORDER BY s.score DESC, target ASC
```

Setelah ditulis, jumlah relasi SIMILAR_TO di database bertambah dari 0 menjadi 5. Hasil query pengecekan:

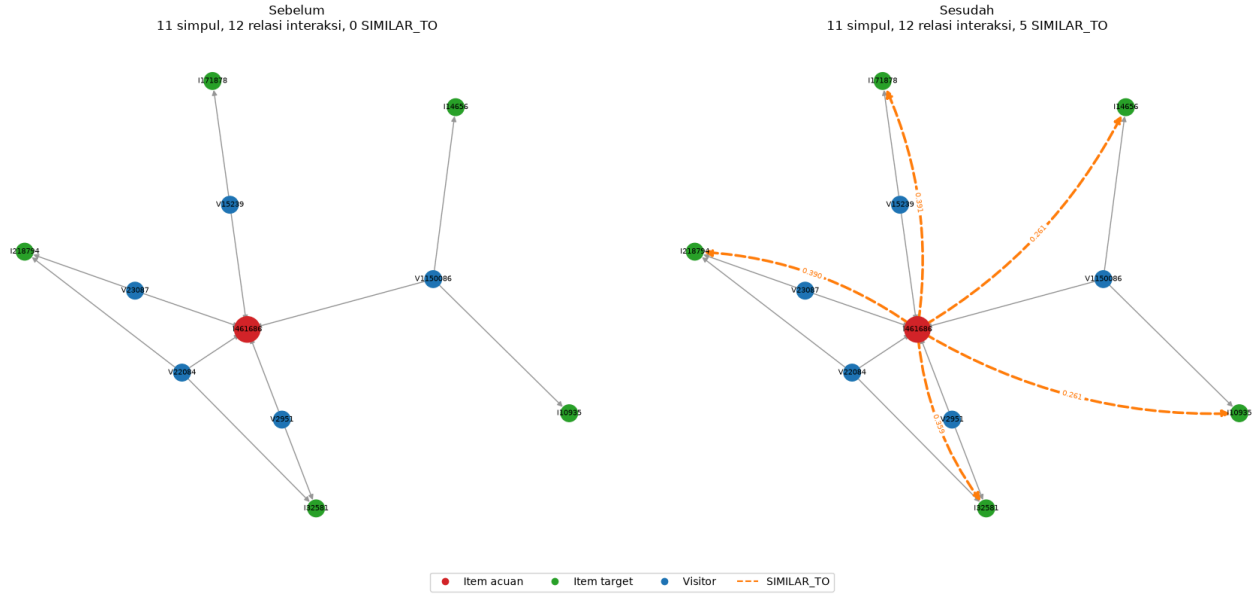
source	target	similarity	method	fold
461686	171878	0,390812	cosine	P1
461686	218794	0,389686	cosine	P1
461686	32581	0,359416	cosine	P1
461686	10935	0,260603	cosine	P1
461686	14656	0,260603	cosine	P1

Soal 8. Graf Sebelum dan Sesudah Similarity

Kondisi graf dilihat dengan query berikut di Neo4j Browser. Bagian pertama menampilkan jalur dari item acuan ke lima item mirip melalui visitor, dan bagian kedua menampilkan relasi SIMILAR_TO. Sebelum similarity ditulis, bagian kedua tidak mengembalikan apa-apa.

```
MATCH p = (ref:Item {itemId: 461686})<-[r1]-(v:Visitor)-[r2]->(o:Item)
WHERE r1.fold = 'P1' AND r2.fold = 'P1' AND o.itemId IN [171878, 218794,
32581, 10935, 14656]
RETURN p LIMIT 30
UNION
MATCH p = (:Item {itemId: 461686})-[:SIMILAR_TO]->(:Item)
RETURN p
```

Pada kondisi sebelum, graf berisi 11 simpul (1 item acuan, 5 item mirip, dan 5 visitor penghubung) dengan 12 relasi interaksi. Visitor penghubungnya adalah 2951, 15239, 22084, 23087, dan 1150086. Pada kondisi sesudah, simpulnya tetap 11 tetapi bertambah 5 relasi SIMILAR_TO dari item 461686.



Dari gambar dapat dilihat bahwa sebelum similarity dihitung, item acuan dan item mirip hanya terhubung secara tidak langsung, yaitu harus melewati visitor terlebih dahulu (dua langkah). Setelah similarity dihitung, kedua item terhubung langsung oleh relasi SIMILAR_TO yang membawa nilai similarity (garis putus-putus oranye). Dengan relasi ini, item yang mirip bisa langsung diambil dari graf tanpa harus menghitung ulang.

B. Analytics

Soal 1. Rekomendasi KNN untuk User Acak

User dipilih secara acak dari kumpulan kandidat yang memenuhi dua syarat, yaitu aktif pada P1 dan P2 serta memiliki minimal 5 item berbeda di P1. Syarat aktif di P2 diperlukan agar hasil rekomendasi bisa dievaluasi, sedangkan syarat 5 item agar riwayat user cukup untuk mencari tetangga. Terdapat 2.099 visitor yang memenuhi syarat. Kandidat diurutkan berdasarkan visitorId, lalu satu visitor diambil memakai random generator numpy dengan seed 42. Karena seed dan urutannya tetap, setiap kali dijalankan akan terpilih user yang sama.

```
items_p1 = train.groupby("visitorid")["itemid"].nunique()
pool = np.sort(np.intersect1d(items_p1[items_p1 >= 5].index,
test["visitorid"].unique()))

rng = np.random.default_rng(42)
TARGET_USER = int(rng.choice(pool))
```

User yang terpilih adalah visitor 133770. Pada P1 user ini berinteraksi dengan 8 item. Item 268006 memiliki bobot 4 karena dilihat dan dimasukkan ke keranjang, sedangkan tujuh item lainnya (35002, 205138, 198615, 237524, 304558, 379301, dan 465582) hanya dilihat dengan bobot 1.

a. Skrip Python KNN

KNN yang dipakai adalah KNN berbasis item pada proyeksi Item. Langkahnya:

1. Untuk setiap item di riwayat user, ambil 5 item tetangga dengan nilai similarity tertinggi.
2. Setiap tetangga diberi skor sebesar bobot interaksi user pada item asal dikali nilai similarity-nya. Jika satu tetangga muncul dari beberapa item asal, skornya dijumlahkan.
3. Urutkan semua tetangga berdasarkan skor dan ambil 3 teratas sebagai item yang diiklankan.

```
def knn_recommend(visitor_id, k=5, n=3):
    user_row = X.getrow(user_index.get_loc(visitor_id))
    scores, sources = {}, {}
    for item_pos, w in zip(user_row.indices, user_row.data):
        neighbors = S.getrow(item_pos)
        order = np.lexsort((item_index[neighbors.indices], -neighbors.data))
[:k]
        for nb_pos, sim in zip(neighbors.indices[order],
neighbors.data[order]):
            nb_item = int(item_index[nb_pos])
            scores[nb_item] = scores.get(nb_item, 0.0) + float(w) *
float(sim)
            sources.setdefault(nb_item, []).append(int(item_index[item_pos]))
    ranking = sorted(scores.items(), key=lambda x: (-x[1], x[0]))[:n]
    return pd.DataFrame(
        [(rank, item, score, sources[item]) for rank, (item, score) in
enumerate(ranking, start=1)],
        columns=["rank", "itemId", "score", "tetangga_dari_item"],
    ).set_index("rank")

recommendations = knn_recommend(133770, k=5, n=3)
```

Item yang sudah pernah dilihat user tetap boleh direkomendasikan, karena menampilkan kembali produk yang pernah dilihat juga umum dilakukan pada iklan toko online.

b. Tiga item yang diiklankan (k = 5)

Peringkat	Item	Skor	Berasal dari item	Pernah dilihat di P1
1	153874	3,1239	268006	Tidak
2	319773	2,7603	268006	Tidak
3	372617	2,6520	268006	Tidak

Ketiga item yang diiklankan berasal dari tetangga item 268006. Hal ini wajar karena item 268006 adalah satu-satunya item yang dimasukkan ke keranjang oleh user, sehingga bobotnya paling besar dan skor tetangganya ikut menjadi paling tinggi. Ketiga item ini juga belum pernah dilihat user sebelumnya.

Soal 2. Evaluasi Rekomendasi

Evaluasi dilakukan dengan membandingkan item rekomendasi dengan item yang benar-benar disentuh user pada P2. Sebuah item dianggap relevan (rel = 1) jika ada di P2, dan tidak relevan (rel = 0) jika tidak ada. Karena rekomendasi hanya 3 item, posisi ke-4 dan ke-5 dianggap kosong.

Pada P2, user 133770 menyentuh 2 item, yaitu 134751 dan 343190.

```
def hit_at_k(recommended, truth, k=5):
    return int(any(item in truth for item in recommended[:k]))

def ndcg_at_k(recommended, truth, k=5):
    dcg = sum(1 / np.log2(i + 2) for i, item in enumerate(recommended[:k]) if
item in truth)
    idcg = sum(1 / np.log2(i + 2) for i in range(min(len(truth), k)))
    return dcg / idcg if idcg > 0 else 0.0
```

Posisi	Item	Ada di P2?	rel	Pembagi log2(posisi+1)	Nilai rel / pembagi
1	153874	Tidak	0	1,0000	0
2	319773	Tidak	0	1,5850	0
3	372617	Tidak	0	2,0000	0
4	kosong	-	0	2,3219	0
5	kosong	-	0	2,5850	0

a. Hit@5

Hit@5 bernilai 1 jika minimal ada satu item relevan di lima posisi teratas, dan 0 jika tidak ada. Karena tidak ada satu pun item rekomendasi yang muncul di P2, maka Hit@5 = 0.

b. NDCG@5

DCG@5 adalah jumlah rel dibagi log2(posisi + 1) untuk posisi 1 sampai 5. Karena semua rel bernilai 0, maka DCG@5 = 0.

IDCG@5 adalah nilai DCG terbaik yang mungkin dicapai. Karena user memiliki 2 item di P2, kondisi terbaiknya adalah kedua item itu berada di posisi 1 dan 2, sehingga IDCG@5 = 1/1 + 1/1,5850 = 1 + 0,6309 = 1,6309.

NDCG@5 = DCG@5 / IDCG@5 = 0 / 1,6309 = 0.

Sebagai pembanding, perhitungan yang sama dilakukan untuk semua 2.099 kandidat user. Rata-rata yang diperoleh adalah Hit@5 = 0,0667 dan NDCG@5 = 0,0331. Artinya hanya sekitar 6,7% user yang mendapat minimal satu rekomendasi yang tepat, sehingga hasil 0 untuk user 133770 masih wajar. Menurut saya rendahnya nilai ini disebabkan oleh data yang sangat jarang (kebanyakan visitor hanya melihat satu item), periode uji yang hanya 3 minggu, dan jumlah rekomendasi yang hanya 3 item.

Kesimpulan

Dengan periode training 25 Mei sampai 13 Agustus 2015 dan periode testing 14 Agustus sampai 3 September 2015, subgraph yang terbentuk berisi 1.065.669 Visitor dan 208.339 Item. Jumlah ini sama antara Neo4j dan file CSV hasil filter. Dari hasil VSM, vektor user sangat kosong karena sekitar 80% visitor hanya menyentuh satu item, sedangkan vektor item lebih berisi. Proyeksi Item juga lebih ringan, sehingga proyeksi Item dipakai untuk menghitung similarity dan rekomendasi.

Item yang paling mirip dengan item 461686 adalah item 171878 dengan similarity 0,3908, dan hasil dari Python serta Cypher sama. Kelima nilai similarity sudah disimpan ke graf sebagai relasi SIMILAR_TO. Rekomendasi KNN untuk user 133770 menghasilkan item 153874, 319773, dan 372617, tetapi belum ada yang sesuai dengan item yang disentuh user di P2, sehingga Hit@5 dan NDCG@5 bernilai 0. Secara keseluruhan rata-rata Hit@5 untuk semua kandidat adalah 0,0667 dan NDCG@5 adalah 0,0331.